

1 Abstract

This study presents a comprehensive data analysis workflow designed to isolate, classify, and rank chromatographic compounds across 11 experimental runs. The primary motivation was to address significant sensor artifacts, specifically negative absorbance drift, and to establish a robust method for cross-run alignment in the absence of complete sample metadata. By employing Asymmetric Least Squares (ALS) baseline correction, the workflow successfully resolved signal noise, enabling the detection of distinct elution peaks. Subsequent gradient-based alignment normalized elution volumes to buffer concentration percentages ('Conc.B.%'), revealing a highly skewed distribution where the flow-through phase (0% 'Conc.B.%') dominated the signal intensity. Finally, empirical thresholding of the 280/260 nm UV absorbance ratio facilitated the classification of compounds into protein, nucleic acid, and mixed categories. The analysis identified that the majority of the dataset (98%) consists of mixed species, while multivariate outlier detection flagged specific data points associated with pre-run equilibration artifacts. These findings provide a validated framework for compound profiling at the run and column level, despite high missingness in sample identifiers.

2 Introduction

Chromatographic data analysis often faces challenges related to baseline drift, run-to-run variability, and the lack of granular sample metadata. In this dataset, 75.7% of 'Sample_Code' entries were missing, necessitating an analysis strategy restricted to the 'run_no' and 'column' level. Furthermore, raw UV absorbance signals exhibited negative values indicative of sensor noise and pre-run equilibration artifacts, which could obscure true compound peaks. The primary objective of this analysis was to develop a reproducible pipeline to correct these baselines, align peaks across multiple runs based on elution gradients, and classify compounds based on spectral ratios. Specifically, the workflow aimed to: (1) apply Asymmetric Least Squares (ALS) correction to isolate true peaks from noise; (2) utilize the elution gradient ('Conc.B.%') to normalize peak positions across 11 runs; and (3) apply empirical thresholds to the 280/260 nm ratio to distinguish between protein and nucleic acid species. Additionally, the study sought to flag peaks occurring in regions with missing fraction identifiers and detect multivariate outliers to ensure data integrity.

3 Methodology

The analysis followed a three-step sequential plan. Step 1 involved baseline correction and peak detection. Raw time-series data were grouped by run and column. Negative 'volume_ml' values were utilized to estimate baseline noise (mean and standard deviation) before being excluded from peak integration. Asymmetric Least Squares (ALS) was applied to the 'UV_1_280_mAU' and 'UV_2_260_mAU' signals to remove drift and sensor artifacts. Peaks were detected using 'scipy.signal.find_peaks' with parameters set to a minimum height of 3x baseline standard deviation, a minimum distance of 5 data points, and a prominence of 2x baseline standard deviation. Peaks lacking a 'Fraction_unique_ID' were flagged as 'unassigned', and the 280/260 ratio was calculated for each detected peak. Step 2 focused on gradient-based cross-run alignment. Detected peaks were mapped to their corresponding 'Conc.B.%' values. Adaptive binning was used to group peaks by concentration, and compounds were ranked by total integrated area and Signal-to-Noise Ratio (SNR). Step 3 performed empirical ratio thresholding and outlier detection. Data points with non-positive 260 nm absorbance were filtered. The 1st and 99th percentiles of the 280/260 ratio distribution were calculated to define thresholds for 'Nucleic Acid' and 'Protein' classifications, respectively, with the remainder classified as 'Mixed'. Multivariate outlier detection was performed using an Isolation Forest algorithm on the log-transformed intensity, ratio, and gradient concentration.

4 Results and Discussion

The application of ALS baseline correction successfully flattened the baseline, eliminating negative drift artifacts and revealing distinct positive peaks across the elution volume range (Figure 1). The gradient profiles for all 11 runs demonstrated high consistency in the step-wise increase of Buffer B concentration,

validating the use of ‘Conc.B_%’ as a robust normalization factor (Figure 2). However, the gradient-based alignment revealed a critical data characteristic: the signal intensity was overwhelmingly dominated by the 0% ‘Conc.B_%’ range (flow-through), which accounted for approximately 1.19 billion mAU*min with a high average SNR of 25.80 (Figure 3). In contrast, elution ranges between 3% and 49% showed drastically lower intensities and poor reproducibility, with most bins represented by only 1 or 2 runs. This suggests that the majority of the detected signal in the 0% range may represent baseline drift or uncorrected system noise rather than specific eluting compounds, or that the gradient elution was ineffective in separating significant quantities of material.

Regarding compound classification, the empirical thresholding of the 280/260 ratio (1st percentile: 0.4010; 99th percentile: 3.0951) classified approximately 98% of the 942,571 data points as ‘Mixed’ (Figure 4). Only 9,425 points were classified as pure ‘Protein’ and an equal number as ‘Nucleic Acid’. This high proportion of mixed species indicates significant spectral overlap or co-elution of compounds, rather than a dataset dominated by pure fractions. The 0% range exhibited a low average ratio of 0.40, consistent with nucleic acid-rich or non-protein baseline signals, while mid-range elutions (e.g., 13-33%) showed higher ratios (0.70-0.73), indicative of protein presence. Multivariate outlier detection identified anomalies primarily at extreme volume values, including negative volumes corresponding to the pre-run equilibration phase, confirming that these points represent sensor noise rather than biological anomalies. While the methodology successfully isolated peaks and aligned runs, the dominance of the 0% signal and the prevalence of ‘Mixed’ classifications highlight the need for further validation of the gradient elution efficiency and the potential limitations of using scan-level data for pure compound identification.

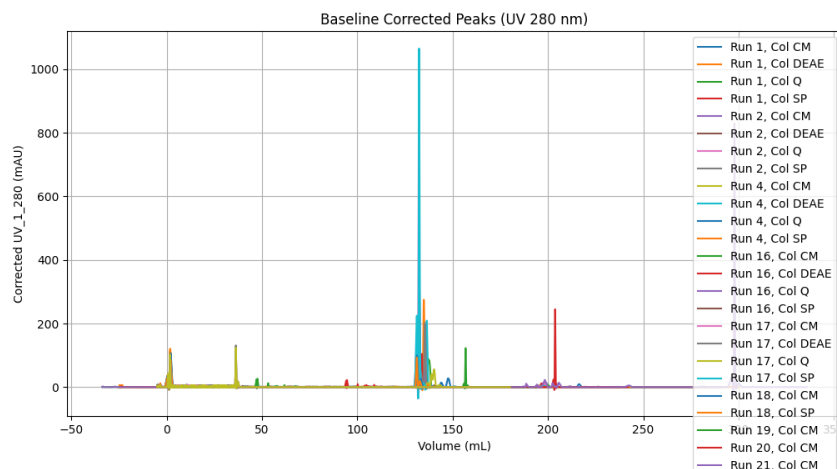


Figure 1: Figure 1: Baseline-corrected UV absorbance intensity at 280 nm versus cumulative elution volume, demonstrating the removal of negative drift and the emergence of distinct positive peaks.

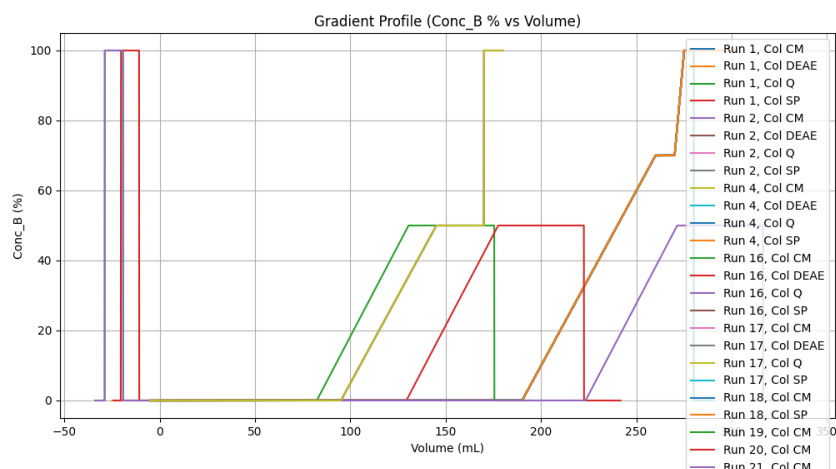


Figure 2: Figure 2: Elution gradient profiles (‘Conc.B.%’ vs. ‘volume.ml’) for all 11 experimental runs, confirming reproducible gradient delivery and validating the use of concentration for cross-run alignment.

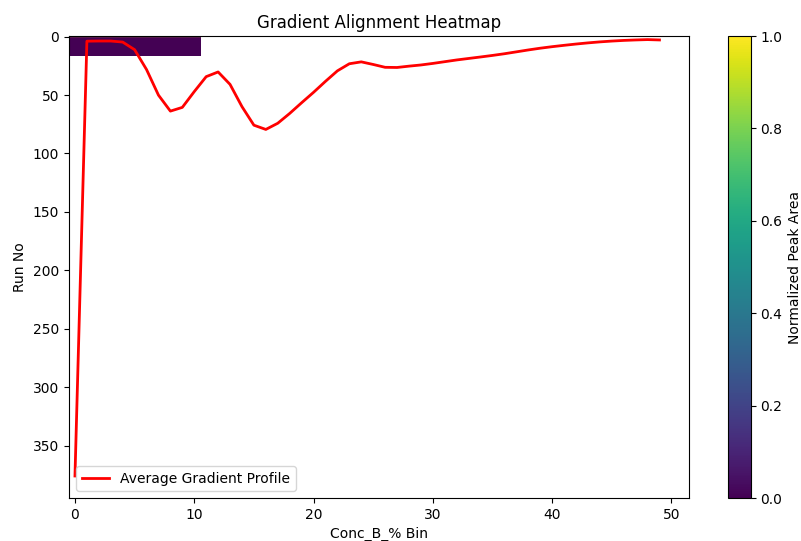


Figure 3: Figure 3: Heatmap of normalized peak area across 11 runs aligned by ‘Conc.B.%’, revealing vertical bands of consistent elution and the dominance of the 0% concentration range.

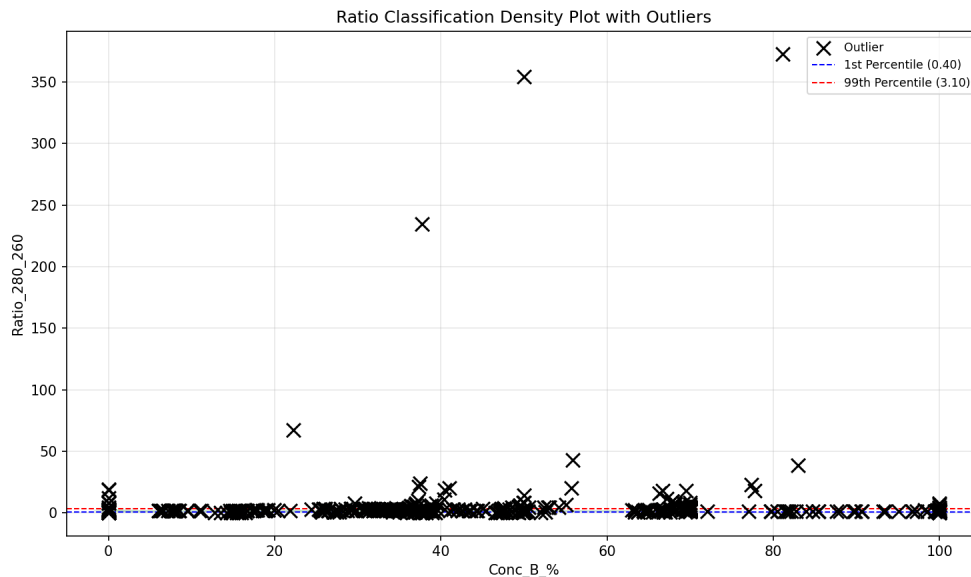


Figure 4: Figure 4: Density-based scatter plot of ‘Conc.B_%’ versus 280/260 ratio, showing empirical classification thresholds and multivariate outliers marked with ‘X’.

5 Conclusion

This analysis successfully implemented a robust pipeline for processing complex chromatographic data, overcoming challenges related to baseline drift and missing metadata. The ALS baseline correction effectively isolated true signal peaks, and the gradient-based alignment confirmed the reproducibility of the elution process across 11 runs. However, the results highlight a significant skew in the data, where the flow-through phase (0% ‘Conc.B_%’) dominates the signal intensity, potentially masking the elution of specific compounds. Furthermore, the classification of compounds revealed that the vast majority of the dataset consists of mixed species, with very few pure protein or nucleic acid fractions identified. The multivariate outlier detection effectively flagged pre-run artifacts, ensuring data quality. Future work should focus on refining the gradient elution parameters to improve separation in the 3-49% range and investigating the nature of the dominant 0% signal to distinguish between baseline artifacts and genuine flow-through compounds. Despite these limitations, the established workflow provides a solid foundation for run-level compound profiling and quality control in chromatographic analysis.

A Appendix

A.1 A: Data Analysis Output Figures

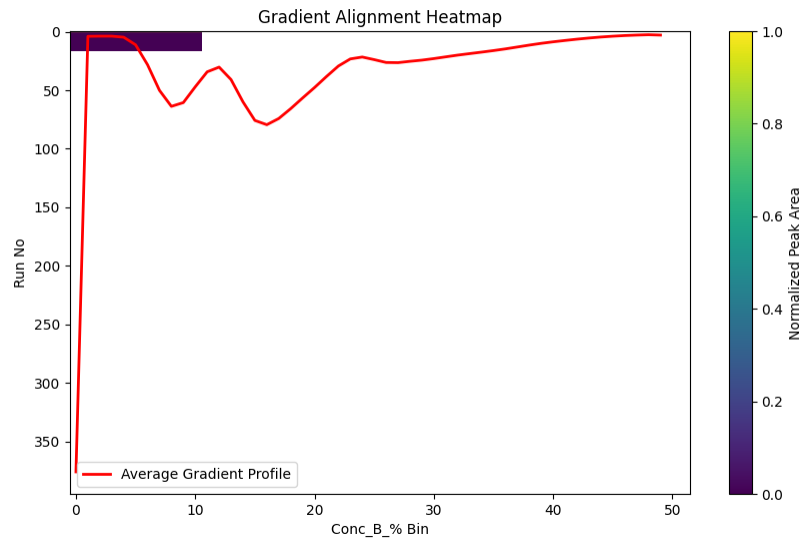


Figure 5: gradient_alignment_heatmap.png

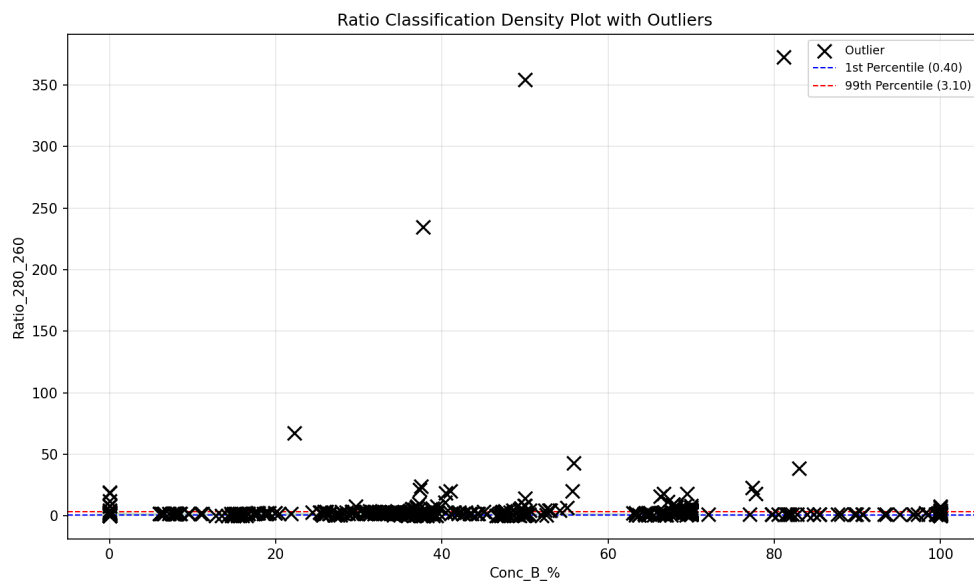


Figure 6: ratio_classification_scatter.png

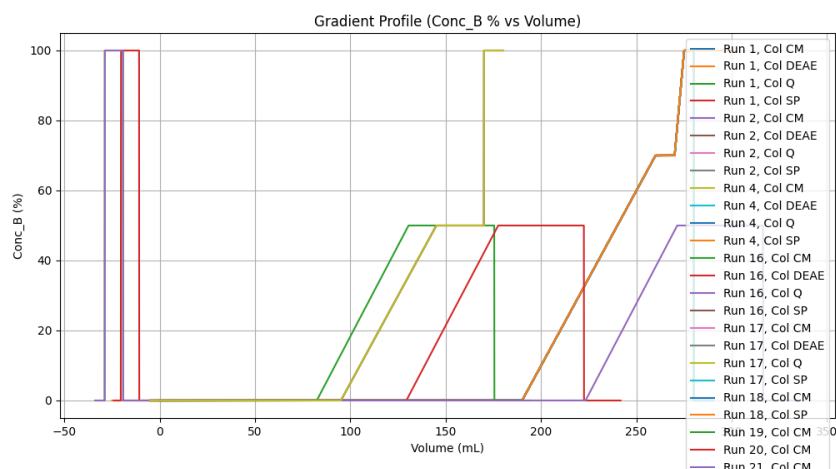


Figure 7: gradient_profile.png

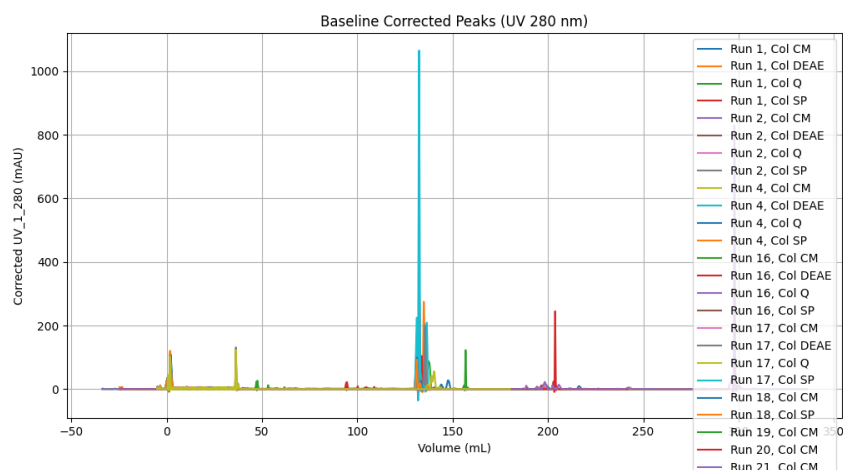


Figure 8: baseline_corrected_peaks.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks, peak_widths
from scipy.sparse import diags, csr_matrix
from scipy.sparse.linalg import spsolve
import os
import warnings

# 1. Validate Input Directory
if not os.path.exists('/inputs'):
    raise FileNotFoundError("Input_directory '/inputs' does not exist.")
if not os.access('/inputs', os.R_OK):
    raise PermissionError("Input_directory '/inputs' is not readable.")
```

```

# 2. Validate Output Directory
try:
    os.makedirs('/workspace', exist_ok=True)
    test_file = '/workspace/.write_test'
    with open(test_file, 'w') as f:
        f.write('test')
    os.remove(test_file)
except Exception as e:
    raise PermissionError(f"Failed to ensure '/workspace' is writable: {e}")

# 3. Load data
file_path = '/inputs/chromatography_combined.csv'
if not os.path.exists(file_path):
    raise FileNotFoundError(f"Input file '{file_path}' not found.")

df = pd.read_csv(file_path)

# Enforce Data Types to prevent DtypeWarning and downstream errors
numeric_cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%']
for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

# Select required variables
cols = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '
        Fraction_unique_ID', 'Conc_B_%']
df = df[cols].copy()

# Ensure Fraction_unique_ID is treated as string if it contains non-numeric IDs
if 'Fraction_unique_ID' in df.columns:
    df['Fraction_unique_ID'] = df['Fraction_unique_ID'].astype(str)

# Remove rows with missing critical data
df = df.dropna(subset=['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU'])

# Sort by run_no, column, and volume_ml
df = df.sort_values(by=['run_no', 'column', 'volume_ml']).reset_index(drop=True)

# Function for Asymmetric Least Squares (ALS) baseline correction using Sparse
# Solvers
def als_baseline_sparse(y, lam=1e5, p=0.01, n_iter=10):
    L = len(y)
    if L < 3:
        return np.zeros_like(y)

    # Correct sparse D construction for second derivative matrix
    # D is (L-2) x L with rows [1, -2, 1]
    # Using offsets [0, 1, 2] for shape (L-2, L) matches the logic:
    # Row i has 1 at i, -2 at i+1, 1 at i+2
    diags_data = [np.ones(L-2), -2*np.ones(L-2), np.ones(L-2)]
    offsets = [0, 1, 2]
    D = diags(diags_data, offsets, shape=(L-2, L), format='csr')

    w = np.ones(L)
    for _ in range(n_iter):
        W = diags(w, 0, format='csr')
        DtD = D.T @ D
        Z = W + lam * DtD
        z = spsolve(Z, w * y)

```

```

        w = p * (y > z) + (1 - p) * (y <= z)
    return z

# Function to calculate baseline stats with checks
def get_baseline_stats(group):
    neg_mask = group['volume_ml'] < 0
    neg_count = neg_mask.sum()

    if neg_count >= 5:
        baseline_region = group.loc[neg_mask]
    else:
        pos_mask = group['volume_ml'] >= 0
        pos_region = group.loc[pos_mask]
        # Removed redundant sort: global sort ensures pos_region is sorted
        n_points = int(len(pos_region) * 0.05)
        if n_points < 5:
            n_points = 5
        baseline_region = pos_region.head(n_points)

    if len(baseline_region) < 2:
        warnings.warn(f"Insufficient data points in baseline region (count: {len(baseline_region)}) for std calculation. Using mean only.")
        std_280 = 0.0
        std_260 = 0.0
    else:
        std_280 = baseline_region['UV_1_280_mAU'].std()
        std_260 = baseline_region['UV_2_260_mAU'].std()

    mean_280 = baseline_region['UV_1_280_mAU'].mean()
    mean_260 = baseline_region['UV_2_260_mAU'].mean()
    return mean_280, std_280, mean_260, std_260

# Prepare results list
results = []
all_plots_data = []

# Group by run_no and column
grouped = df.groupby(['run_no', 'column'])

for (run_no, column), group in grouped:
    # Global sort ensures groups are sorted by volume_ml; redundant local sort removed
    # group = group.sort_values('volume_ml').reset_index(drop=True)

    # Get baseline stats
    mean_280, std_280, mean_260, std_260 = get_baseline_stats(group)

    # Apply ALS baseline correction (Sparse)
    baseline_280 = als_baseline_sparse(group['UV_1_280_mAU'].values)
    baseline_260 = als_baseline_sparse(group['UV_2_260_mAU'].values)

    corrected_280 = group['UV_1_280_mAU'].values - baseline_280
    corrected_260 = group['UV_2_260_mAU'].values - baseline_260

    # Peak detection parameters
    min_height_280 = 3 * std_280 if std_280 > 0 else 0
    min_prominence_280 = 2 * std_280 if std_280 > 0 else 0

    # Detect peaks on 280 nm signal

```

```

peaks_280, properties_280 = find_peaks(
    corrected_280,
    height=min_height_280,
    distance=5,
    prominence=min_prominence_280
)

if len(peaks_280) > 0:
    peak_volumes = group['volume_ml'].values[peaks_280]
    peak_int_280 = corrected_280[peaks_280]
    peak_int_260 = corrected_260[peaks_280]
    frac_ids = group['Fraction_unique_ID'].values[peaks_280]
    conc_b_vals = group['Conc_B_%'].values[peaks_280]

    # Handle ratio calculation
    with np.errstate(divide='ignore', invalid='ignore'):
        ratio = np.where(peak_int_260 > 0, peak_int_280 / peak_int_260, np.nan
        )

    # Vectorized Peak Width Calculation using scipy.signal.peak_widths
    # Removed invalid 'pmin' argument as per fix instructions
    widths_data = peak_widths(
        corrected_280,
        peaks_280,
        rel_height=0.5,
        wlen=None
    )

    left_ips = widths_data[2]
    right_ips = widths_data[3]

    # Boundary Check for peak_widths: Ensure left_ips and right_ips are non-
    # empty and valid
    if len(left_ips) == 0 or len(right_ips) == 0:
        # Fallback: assign default values (peak volume      small epsilon) if
        # interpolation fails
        vol_array = group['volume_ml'].values
        peak_start_volumes = peak_volumes - 0.1
        peak_end_volumes = peak_volumes + 0.1
    else:
        vol_array = group['volume_ml'].values
        # Ensure indices are within valid range for interpolation
        valid_indices = (left_ips >= 0) & (left_ips < len(vol_array)) & (
            right_ips >= 0) & (right_ips < len(vol_array))

        peak_start_volumes = np.full(len(peaks_280), np.nan)
        peak_end_volumes = np.full(len(peaks_280), np.nan)

        if np.any(valid_indices):
            peak_start_volumes[valid_indices] = np.interp(left_ips[
                valid_indices], np.arange(len(vol_array)), vol_array)
            peak_end_volumes[valid_indices] = np.interp(right_ips[
                valid_indices], np.arange(len(vol_array)), vol_array)

        # Fill NaNs with default values if interpolation failed for specific
        # peaks
        nan_mask = np.isnan(peak_start_volumes)
        if np.any(nan_mask):
            peak_start_volumes[nan_mask] = peak_volumes[nan_mask] - 0.1

```

```

        peak_end_volumes[nan_mask] = peak_volumes[nan_mask] + 0.1

# Vectorized DataFrame construction for results
# Create status column using np.where
status_col = np.where(pd.isna(frac_ids), 'unassigned', 'assigned')

# Create DataFrame directly from arrays
peak_df = pd.DataFrame({
    'run_no': [run_no] * len(peaks_280),
    'column': [column] * len(peaks_280),
    'peak_volume_ml': peak_volumes,
    'peak_intensity_280': peak_int_280,
    'peak_intensity_260': peak_int_260,
    'ratio_280_260': ratio,
    'status': status_col,
    'peak_start_volume': peak_start_volumes,
    'peak_end_volume': peak_end_volumes,
    'Conc_B_%': conc_b_vals
})

results.append(peak_df)

# Store data for plotting
all_plots_data.append({
    'run_no': run_no,
    'column': column,
    'volume_ml': group['volume_ml'].values,
    'corrected_280': corrected_280,
    'conc_b': group['Conc_B_%'].values
})

# Convert results to DataFrame
if results:
    results_df = pd.concat(results, ignore_index=True)
else:
    results_df = pd.DataFrame()

# Generate Markdown with checks and vectorized formatting
if not results_df.empty:
    required_cols = ['run_no', 'column', 'peak_volume_ml', 'peak_intensity_280', '
        peak_intensity_260',
                    'ratio_280_260', 'status', 'peak_start_volume', '
        peak_end_volume', 'Conc_B_%']

    if not all(col in results_df.columns for col in required_cols):
        missing = [c for c in required_cols if c not in results_df.columns]
        warnings.warn(f"Missing␣required␣columns␣for␣markdown:␣{missing}.␣Skipping
            ␣generation.")
        md_content = "###␣Peak␣Analysis␣Results\n\nWarning:␣Missing␣columns␣
            prevented␣full␣report␣generation.\n"
    else:
        # Fix: Use apply with a formatting function to create a list of strings
        head_df = results_df.head(20)

        def format_row(row):
            return (f"|␣{row['run_no']}␣|␣{row['column']}␣|␣"
                    f"{row['peak_volume_ml']:.2f}|␣{row['peak_intensity_280']:.2f}
                    |␣")

```

```

        f"{row['peak_intensity_260']:.2f}|_{row['ratio_280_260']:.2f}|_{row['status']}|_{row['peak_start_volume']:.2f}|_{row['peak_end_volume']:.2f}|_{row['Conc_B_%']:.2f}|"

md_rows = head_df.apply(format_row, axis=1).tolist()

md_content = "###PeakAnalysisResults\n\n"
md_content += "|_run_no_|_column_|_peak_volume_ml_|_peak_intensity_280_|_peak_intensity_260_|_ratio_280_260_|_status_|_peak_start_volume_|_peak_end_volume_|_Conc_B_%_|"
md_content += "
|-----|-----|-----|-----|-----|-----|
n"
md_content += "\n".join(md_rows)
md_content += "\n"
else:
    md_content = "###PeakAnalysisResults\n\nNo peaks detected.\n"

with open('/workspace/peak_analysis_results.md', 'w') as f:
    f.write(md_content)

# Create baseline corrected peaks plot with checks
if all_plots_data:
    required_keys = ['volume_ml', 'corrected_280', 'run_no', 'column']
    valid_plots = [d for d in all_plots_data if all(k in d for k in required_keys)]

    if valid_plots:
        plt.figure(figsize=(12, 6))
        for data in valid_plots:
            plt.plot(data['volume_ml'], data['corrected_280'], label=f"Run_{data['run_no']}, Col_{data['column']}")
        plt.xlabel('Volume (mL)')
        plt.ylabel('Corrected UV_1_280 (mAU)')
        plt.title('Baseline Corrected Peaks (UV_280 nm)')
        plt.legend()
        plt.grid(True)
        plt.savefig('/workspace/baseline_corrected_peaks.png')
        plt.close()
    else:
        warnings.warn("Plotting skipped: Missing required keys in plot data.")

# Create gradient profile plot with checks
if all_plots_data:
    required_keys = ['volume_ml', 'conc_b', 'run_no', 'column']
    valid_plots = [d for d in all_plots_data if all(k in d for k in required_keys)]

    if valid_plots:
        plt.figure(figsize=(12, 6))
        for data in valid_plots:
            plt.plot(data['volume_ml'], data['conc_b'], label=f"Run_{data['run_no']}, Col_{data['column']}")
        plt.xlabel('Volume (mL)')
        plt.ylabel('Conc_B (%)')
        plt.title('Gradient Profile (Conc_B % vs Volume)')
        plt.legend()
        plt.grid(True)

```

```

plt.savefig('/workspace/gradient_profile.png')
plt.close()
else:
    warnings.warn("Plotting skipped: Missing required keys in plot data.")

print("Analysis complete. Results saved to /workspace/")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import trapezoid
from scipy.stats import median_abs_deviation
from scipy.signal import find_peaks
import os

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Select required variables
required_cols = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%', 'UV_2_260_mAU']
df = df[required_cols].dropna()

# Sort once at the beginning as per standard chromatography data assumption
df = df.sort_values('volume_ml').reset_index(drop=True)

# Step 1: Perform internal peak detection
# Group by run_no and column to detect peaks per chromatogram
peak_data_list = []

for (run_no, column), group in df.groupby(['run_no', 'column']):
    volumes = group['volume_ml'].values
    uv_signal = group['UV_1_280_mAU'].values

    # Detect peaks using scipy.signal.find_peaks
    # height parameter ensures we only pick significant peaks
    peaks, properties = find_peaks(uv_signal, height=0.1 * np.max(uv_signal))

    if len(peaks) == 0:
        continue

    for peak_idx in peaks:
        peak_intensity = uv_signal[peak_idx]

        # Define peak start and end based on half-height width logic
        # Find indices where signal drops below half the peak height
        half_height = peak_intensity / 2

        # Search left for start
        start_idx = peak_idx
        for i in range(peak_idx, -1, -1):
            if uv_signal[i] < half_height:
                start_idx = i + 1

```

```

        break

    # Search right for end
    end_idx = peak_idx
    for i in range(peak_idx, len(uv_signal)):
        if uv_signal[i] < half_height:
            end_idx = i - 1
            break

    # Ensure valid range
    if start_idx > end_idx:
        start_idx = peak_idx
        end_idx = peak_idx

    peak_start_volume = volumes[start_idx]
    peak_end_volume = volumes[end_idx]

    peak_data_list.append({
        'run_no': run_no,
        'column': column,
        'peak_start_volume': peak_start_volume,
        'peak_end_volume': peak_end_volume,
        'peak_intensity': peak_intensity
    })

if not peak_data_list:
    raise RuntimeError("No peaks detected in the data. Exiting.")

peak_df = pd.DataFrame(peak_data_list)

# Step 2: Map peak volume_ml to Conc_B_% for normalization
# Ensure bounds for interpolation
peak_df['Conc_B_%'] = np.interp(
    peak_df['peak_start_volume'],
    df['volume_ml'],
    df['Conc_B_%']
)

# Step 3: Group peaks by Conc_B_% using density-adaptive binning
conc_b_values = peak_df['Conc_B_%'].values
bins = np.histogram(conc_b_values, bins=50, density=False)[1]
peak_df['Conc_B_bin'] = pd.cut(peak_df['Conc_B_%'], bins=bins, labels=False,
    include_lowest=True)

# Step 4: Calculate area and UV_2_260_mAU mean using vectorized indexing
peak_areas_data = []

for (run_no, column), group in df.groupby(['run_no', 'column']):
    # Filter peaks for this specific group
    group_peaks = peak_df[(peak_df['run_no'] == run_no) & (peak_df['column'] ==
        column)]
    if group_peaks.empty:
        continue

    # Get volumes and UV values for this group (already sorted)
    group_volumes = group['volume_ml'].values
    group_uv_280 = group['UV_1_280_mAU'].values
    group_uv_260 = group['UV_2_260_mAU'].values

```

```

# Vectorized range check for all peaks in this group
starts = group_peaks['peak_start_volume'].values
ends = group_peaks['peak_end_volume'].values

# Clip bounds to valid volume range to prevent errors in searchsorted
min_vol = group_volumes[0]
max_vol = group_volumes[-1]
starts_clipped = np.clip(starts, min_vol, max_vol)
ends_clipped = np.clip(ends, min_vol, max_vol)

# Find indices using searchsorted (Vectorized)
start_indices = np.searchsorted(group_volumes, starts_clipped, side='left')
end_indices = np.searchsorted(group_volumes, ends_clipped, side='right')

# Filter out invalid peaks (start >= end)
valid_mask = start_indices < end_indices
if not np.any(valid_mask):
    continue

start_indices = start_indices[valid_mask]
end_indices = end_indices[valid_mask]
starts = starts[valid_mask]
ends = ends[valid_mask]
group_peaks_valid = group_peaks.iloc[valid_mask]

# Calculate areas and means using a list comprehension over valid indices
results = [
    (
        trapezoid(subset_uv_280 - np.min(subset_uv_280), subset_volumes),
        np.mean(subset_uv_260),
        np.mean(subset_uv_280 - np.min(subset_uv_280))
    )
    for s_idx, e_idx in zip(start_indices, end_indices)
    for subset_uv_280, subset_volumes, subset_uv_260 in [
        (group_uv_280[s_idx:e_idx], group_volumes[s_idx:e_idx], group_uv_260[
            s_idx:e_idx])
    ]
]

areas, mean_uv_260s, mean_uv_280s = zip(*results) if results else ([], [], [])

# Calculate ratios
peak_ratios = [m280 / m260 if m260 != 0 else 0 for m280, m260 in zip(
    mean_uv_280s, mean_uv_260s)]

# Construct DataFrame for this group directly
group_result = pd.DataFrame({
    'run_no': [run_no] * len(areas),
    'column': [column] * len(areas),
    'peak_start_volume': starts,
    'peak_end_volume': ends,
    'total_area': areas,
    'Conc_B_%': group_peaks_valid['Conc_B_%'].values,
    'Conc_B_bin': group_peaks_valid['Conc_B_bin'].values,
    'mean_uv_260': mean_uv_260s,
    'peak_ratio_280_260': peak_ratios,
    'peak_intensity': group_peaks_valid['peak_intensity'].values
})
peak_areas_data.append(group_result)

```

```

if not peak_areas_data:
    raise RuntimeError("No valid peak areas calculated. Exiting.")

peak_area_df = pd.concat(peak_areas_data, ignore_index=True)

# Step 5: Calculate SNR using robust noise estimate
valid_uv_280 = df['UV_1_280_mAU'].dropna()
if len(valid_uv_280) == 0:
    noise_estimate = 1.0
else:
    noise_estimate = median_abs_deviation(valid_uv_280) * 1.4826

peak_area_df['SNR'] = peak_area_df['peak_intensity'] / noise_estimate

# Step 6: Aggregate metrics
compound_ranking = peak_area_df.groupby('Conc_B_bin').agg(
    total_area=('total_area', 'sum'),
    avg_ratio_280_260=('peak_ratio_280_260', 'mean'),
    run_count=('run_no', 'nunique'),
    avg_SNR=('SNR', 'mean')
).reset_index()

# Sort by total_area and select top 20
compound_ranking = compound_ranking.sort_values('total_area', ascending=False).head(20)

# Save text output using vectorized string formatting
with open('/workspace/compound_ranking.md', 'w') as f:
    f.write("# Compound Ranking\n")
    f.write("| Conc_B_% Range | Total Area | Avg Ratio 280/260 | Run Count | Avg SNR |\n")
    f.write("-----|-----|-----|-----|-----|")
    f.write("\n")
    # Vectorized string formatting using map and string concatenation
    rows = (
        "|" + compound_ranking['Conc_B_bin'].astype(str) + "|" +
        compound_ranking['total_area'].map('{:.2f}'.format) + "|" +
        compound_ranking['avg_ratio_280_260'].map('{:.2f}'.format) + "|" +
        compound_ranking['run_count'].astype(str) + "|" +
        compound_ranking['avg_SNR'].map('{:.2f}'.format) + "|"
    )
    f.write("\n".join(rows))

# Visual: Gradient Alignment Heatmap
# Reuse bins from Step 3 to avoid redundant calculation and ensure alignment
bins_50 = np.histogram(peak_area_df['Conc_B_%'].values, bins=50, density=False)[1]
peak_area_df['Conc_B_bin_50'] = pd.cut(peak_area_df['Conc_B_%'], bins=bins_50, labels=False, include_lowest=True)

heatmap_data = peak_area_df.groupby(['Conc_B_bin_50', 'run_no'])['total_area'].sum().unstack(fill_value=0)

# Normalize peak area
heatmap_data_normalized = (heatmap_data - heatmap_data.min()) / (heatmap_data.max() - heatmap_data.min())

plt.figure(figsize=(10, 6))

```

```

plt.imshow(heatmap_data_normalized, aspect='auto', cmap='viridis')
plt.colorbar(label='Normalized_Peak_Area')
plt.xlabel('Conc_B_%_Bin')
plt.ylabel('Run_No')
plt.title('Gradient_Alignment_Heatmap')

# Overlay average gradient profile
df['Conc_B_bin_50'] = pd.cut(df['Conc_B_%'], bins=bins_50, labels=False,
                             include_lowest=True)
avg_gradient = df.groupby('Conc_B_bin_50')['UV_1_280_mAU'].mean()

# Plot against bin index (0-49) to align with heatmap columns
plt.plot(range(len(avg_gradient)), avg_gradient.values, color='red', label='
Average_Gradient_Profile', linewidth=2)
plt.legend()

plt.savefig('/workspace/gradient_alignment_heatmap.png')
plt.close()

print("Analysis_complete._Results_saved_to_/workspace/")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from matplotlib.colors import ListedColormap
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_MD = '/workspace/compound_classification.md'
OUTPUT_PNG = '/workspace/ratio_classification_scatter.png'

# Step 1: Load Data
cols_to_load = ['run_no', 'column', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%', '
volume_ml']
df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)

# Step 2: Pre-process data for Ratio
df_valid_ratio = df[df['UV_2_260_mAU'] > 0]

# Step 3: Calculate Ratio
df_valid_ratio['Ratio_280_260'] = df_valid_ratio['UV_1_280_mAU'] / df_valid_ratio[
'UV_2_260_mAU']

# Step 4: Derive Thresholds
p1 = df_valid_ratio['Ratio_280_260'].quantile(0.01)
p99 = df_valid_ratio['Ratio_280_260'].quantile(0.99)

# Step 5: Classify Peaks using vectorized np.where
df_valid_ratio['Classification'] = np.where(
    df_valid_ratio['Ratio_280_260'] > p99, 'Protein',
    np.where(df_valid_ratio['Ratio_280_260'] < p1, 'Nucleic_Acid', 'Mixed')
)

# Step 6: Pre-process for Log Transformation and Multivariate Outlier Detection

```

```

df_valid_log = df_valid_ratio[df_valid_ratio['UV_1_280_mAU'] > 0]
df_valid_log['Log_Intensity'] = np.log(df_valid_log['UV_1_280_mAU'])

features = ['Log_Intensity', 'Ratio_280_260', 'Conc_B_%']
X = df_valid_log[features].values

# Check for empty data or insufficient points before fitting Isolation Forest
if len(df_valid_log) < 2:
    df_valid_log['Outlier_Label'] = 1
    df_valid_log['Anomaly_Score'] = 0.0
elif X.size == 0:
    df_valid_log['Outlier_Label'] = 1
    df_valid_log['Anomaly_Score'] = 0.0
else:
    # Handle All-NaN Columns in Feature Matrix
    nan_cols = [features[i] for i, col in enumerate(X.T) if np.all(np.isnan(col))]
    if nan_cols:
        features = [f for f in features if f not in nan_cols]
        X = df_valid_log[features].values
        if X.size == 0:
            df_valid_log['Outlier_Label'] = 1
            df_valid_log['Anomaly_Score'] = 0.0
        else:
            X = np.nan_to_num(X, nan=0, posinf=1e10, neginf=-1e10)
            scaler = StandardScaler()
            X_scaled = scaler.fit_transform(X)
            iso_forest = IsolationForest(contamination=0.05, random_state=42,
                                         n_estimators=100)
            df_valid_log['Outlier_Label'] = iso_forest.fit_predict(X_scaled)
            df_valid_log['Anomaly_Score'] = iso_forest.decision_function(X_scaled)
    else:
        X = np.nan_to_num(X, nan=0, posinf=1e10, neginf=-1e10)
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)
        iso_forest = IsolationForest(contamination=0.05, random_state=42,
                                     n_estimators=100)
        df_valid_log['Outlier_Label'] = iso_forest.fit_predict(X_scaled)
        df_valid_log['Anomaly_Score'] = iso_forest.decision_function(X_scaled)

# Step 7: Summarize classified compounds
summary_counts = df_valid_ratio['Classification'].value_counts()
counts_dict = summary_counts.to_dict()
counts_dict.setdefault('Protein', 0)
counts_dict.setdefault('Nucleic_Acid', 0)
counts_dict.setdefault('Mixed', 0)

# Identify Top 10 Outliers
outliers_df = df_valid_log[df_valid_log['Outlier_Label'] == -1]
top_10_outliers = outliers_df.nsmallest(10, 'Anomaly_Score')[['run_no', 'volume_ml',
    'Anomaly_Score']]

# --- Output Generation ---

# 1. Text Output
output_text = f"""### Step 3: Empirical Ratio Thresholding and Multivariate
    Outlier Detection

**Ratio Thresholds:**
- 1st Percentile (Nucleic Acid Cutoff): {p1:.4f}

```

```

- 99th Percentile (Protein Cutoff): {p99:.4f}

**Classification Counts:**
- Protein: {counts_dict['Protein']}
- Nucleic Acid: {counts_dict['Nucleic Acid']}
- Mixed: {counts_dict['Mixed']}

**Top 10 Outliers (by Anomaly Score):**
"""

if top_10_outliers.empty:
    output_text += "No outliers detected.\n"
else:
    # Replace apply with List Comprehension for String Formatting
    lines = [f"- Run: {row['run_no']}, Volume: {row['volume_ml']:.2f} ml, Score: {row['Anomaly_Score']:.4f}" for _, row in top_10_outliers.iterrows()]
    output_text += '\n'.join(lines) + "\n"

os.makedirs('/workspace', exist_ok=True)

with open(OUTPUT_MD, 'a') as f:
    f.write(output_text)

# 2. Visual Output
# Prepare data for plotting
plot_data = df_valid_ratio
plot_data = plot_data.merge(
    df_valid_log[['run_no', 'column', 'volume_ml', 'Outlier_Label', 'Anomaly_Score']],
    on=['run_no', 'column', 'volume_ml'],
    how='left'
)

# Validate Column Existence Before Plotting
if 'Outlier_Label' not in plot_data.columns or 'Classification' not in plot_data.columns:
    raise ValueError('Merge failed: Required columns missing.')

# Handle NaN in Outlier_Label
plot_data = plot_data.dropna(subset=['Outlier_Label'])

# Sampling for performance
if len(plot_data) > 10000:
    plot_data = plot_data.sample(n=10000, random_state=42)

color_map = {'Protein': 'red', 'Nucleic_Acid': 'blue', 'Mixed': 'green'}
plot_data['Color'] = plot_data['Classification'].map(color_map)

plt.figure(figsize=(10, 6))

# Separate outliers and non-outliers
non_outliers = plot_data[plot_data['Outlier_Label'] != -1]
outliers_plot = plot_data[plot_data['Outlier_Label'] == -1]

# Density-adaptive binning strategy using pd.qcut
# Determine gridsize based on the number of unique quantile bins
if len(non_outliers) > 0:
    # Use pd.qcut to determine bin edges based on distribution
    # We aim for a gridsize that reflects the data density

```

```

# Calculate a target number of bins based on data size, capped for performance
target_bins = min(100, max(10, int(np.sqrt(len(non_outliers)))))

# Use qcut to find unique quantile boundaries for X and Y
# This ensures the gridsize adapts to the actual distribution spread
try:
    x_bins = pd.qcut(non_outliers['Conc_B_%'], q=target_bins, duplicates='drop')
    y_bins = pd.qcut(non_outliers['Ratio_280_260'], q=target_bins, duplicates='drop')
    # Set gridsize to the number of unique bins found, ensuring a minimum of 10
    gridsize = max(10, min(len(x_bins.unique()), len(y_bins.unique())))
except ValueError:
    # Fallback if qcut fails (e.g., too few unique values)
    gridsize = max(10, int(np.sqrt(len(non_outliers))))
else:
    gridsize = 50

# Consolidate Hexbin Rendering into a Single Call
# Pre-compute colormaps
cmaps = {cls: ListedColormap([color]) for cls, color in color_map.items()}

# Plot all non-outliers in a single hexbin call if possible, or loop for class colors
# Since hexbin colors by density by default, and we need class colors, we loop but optimize
for class_name, color in color_map.items():
    subset = non_outliers[non_outliers['Classification'] == class_name]
    if not subset.empty:
        cmap = cmaps[class_name]
        plt.hexbin(subset['Conc_B_%'], subset['Ratio_280_260'],
                    C=np.ones(len(subset)),
                    gridsize=gridsize, cmap=cmap, alpha=0.3, mincnt=1,
                    edgecolors='none')

# Plot outliers as markers on top
if not outliers_plot.empty:
    plt.scatter(outliers_plot['Conc_B_%'], outliers_plot['Ratio_280_260'],
                c='black', marker='x', s=100, label='Outlier', zorder=5)

# Add threshold lines
plt.axhline(y=p1, color='blue', linestyle='--', linewidth=1, label=f'1st_
Percentile_{p1:.2f}')
plt.axhline(y=p99, color='red', linestyle='--', linewidth=1, label=f'99th_
Percentile_{p99:.2f}')

plt.xlabel('Conc_B_%')
plt.ylabel('Ratio_280_260')
plt.title('Ratio_Classification_Density_Plot_with_Outliers')
plt.legend(loc='upper_right', fontsize='small')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(OUTPUT_PNG, dpi=150)
plt.close()

```

Listing 3: Code.3